SGA — Sistema de Gestão de Ambientes

Plano de Execução e Evidências do Projeto Integrador II

Escopo Temático: DevOps, Ciência de Dados e Computação em Nuvem | **Versão:** 1.1.0 Final

Equipe TechFlow: Vitor Fazano, Fabio Bitencourt, Guilherme Leite, Ronaldo Soares, Edilaine Paulino, Bianca Cirilo

Repositório: https://github.com/edilainesolde/SGA | **Data:** Novembro de 2026

---

1. Definição do Problema, Usuários e Escopo

Contexto: A gestão de ambientes físicos (salas teóricas, laboratórios de informática/redes e auditórios) em instituições de ensino técnico e superior é historicamente afetada pela descentralização e dispersão de controles em planilhas eletrônicas independentes.

Dores e Fragilidades Identificadas:

- **Conflito de Horários:** Sobreposição de turmas no mesmo espaço físico por ausência de validação atômica;
- **Ociosidade de Recursos:** Falta de visibilidade em tempo real sobre salas desocupadas para reservas emergenciais de docentes;
- **Lentidão na Manutenção:** Falhas estruturais, elétricas e de climatização demoram dias para chegar ao conhecimento da equipe técnica;
- **Ausência de Métricas:** Inexistência de indicadores consolidados de taxa de ocupação, SLA de reparo e tempo médio de solução (MTTR).

Usuários e Partes Interessadas:

Ator / Perfil	Responsabilidade Operacional	Valor Entregue pela Solução
👔 **Coordenação**	Atribuição de disciplinas a docentes e alocação de ambientes	Eliminação de conflitos de grade e visão analítica de ocupação
🗂️ **Secretaria**	Cadastro de cursos, turmas, docentes e relatórios de uso	Centralização de registros mestres e integridade referencial
👩‍🏫 **Docente**	Consulta de grade e reserva pontual de salas disponíveis	Verificação instantânea de disponibilidade com bloqueio inteligente
🔧 **Manutenção**	Triagem, validação de solicitações e execução de ordens	Separação entre sugestão e ocorrência oficial, monitorando SLA
🌐 **Público Geral**	Reporte de defeitos e necessidades de reparo via smartphone	Canal aberto ágil sem necessidade de login ou burocracia

Critérios de Aceite Formais do Projeto:

- **CA-01:** Ingestão e persistência relacional de turmas e ambientes com integridade referencial garantida;

- **CA-02:** Rejeição imediata de arquivos ou cadastros com horários finais invertidos, capacidades negativas ou duplicidades;
- **CA-03:** Bloqueio automático de reservas para salas marcadas como "Ocupada" ou "Em Manutenção";
- **CA-04:** Uma sugestão pública de manutenção só se converte em ordem de serviço após aprovação formal na triagem técnica;
- **CA-05:** Pipeline de CI/CD automatizado no GitHub Actions executando validações de dados e suíte de testes unitários;
- **CA-06:** Execução reproduzível local via contêineres Docker (docker-compose) e endpoint de healthcheck ativo.

2. Planejamento de Dados, Fluxo e Arquitetura

Fluxo Mínimo Integrado Ponta a Ponta:

Fonte de Dados (JSON / CSV / Telemetria) → **Ingestão & Validação** (validador_dados.js) → **Tratamento & Armazenamento** (PostgreSQL / Supabase) → **Análise & Indicadores** (indicadores.js) → **API REST** (Express) → **Aplicação Web** (React SPA na Vercel) → **Observabilidade** (/api/health)

Esquema Relacional Normalizado (PostgreSQL):

Tabela	Chave Primária	Relacionamentos / Chaves Estrangeiras	Regras de Integridade
`ambientes`	id (bigint)	1:N com `alocacoes_ambiente`, 1:N com `sugestoes_manutencao`	codigo único, capacidade > 0, status restrito
`turmas`	id (bigint)	N:1 com `cursos`, 1:N com `alocacoes_ambiente`	codigo único, horario_fim > horario_inicio
`docentes`	id (bigint)	1:N com `turma_componentes_docentes`	registro funcional único
`alocacoes_ambiente`	id (bigint)	N:1 com `ambientes`, `turmas`, `docentes`	Bloqueio de horários colidentes no mesmo dia
`sugestoes_manutencao`	id (bigint)	N:1 com `ambientes`	Status: PENDENTE_ANALISE, APROVADA, NAO_APROVADA, FINALIZADA

3. Ciência de Dados e Indicadores Estratégicos

O SGA implementa e documenta 4 indicadores matemáticos para suporte à tomada de decisão:

Indicador 1: Taxa de Ocupação de Ambientes (TOA)

```
TOA (%) = ( Total de Horas Semanais Alocadas / Capacidade Horária da Grade Semanal ) * 100
```

Fonte dos Dados: `alocacoes_ambiente` e `ambientes` (data/sample/alocacoes.json).

Interpretação Gerencial: Taxas > 85% indicam saturação do espaço; taxas < 50% indicam ociosidade crônica, sinalizando à coordenação oportunidade de consolidar turmas para redução de custos com climatização.

Indicador 2: Taxa de Cumprimento de SLA de Manutenção (T_SLA)

```
SLA (%) = ( Ocorrências Concluídas Dentro do Prazo Limite / Total de Ocorrências Concluídas ) * 100
```

Fonte dos Dados: `sugestoes_manutencao` (campos `finalizado_em`, `analisado_em` e `sla_horas_limite`).

Interpretação Gerencial: Meta institucional ≥ 90%. Níveis inferiores a 75% exigem remanejamento de mão de obra técnica para atendimento de falhas emergenciais (ex: vazamentos e falhas elétricas).

Indicador 3: MTTR — Mean Time To Repair (Tempo Médio de Reparo em Horas)

```
MTTR = Soma( Timestamp Conclusão - Timestamp Aprovação Triagem ) / Total de Reparos
```

Interpretação Gerencial: Mensura exclusivamente o tempo de atuação técnica da equipe de manutenção, desconsiderando o tempo prévio em que a solicitação pública esteve aguardando triagem.

Indicador 4: Taxa de Conversão de Sugestões em Ocorrências (T_CSO)

```
Conversão (%) = ( Sugestões Aprovadas na Triagem / Total de Sugestões Avaliadas ) * 100
```

Interpretação Gerencial: Aferição da qualidade e da pertinência dos reportes enviados pelo público em geral.

4. DevOps, CI/CD e Computação em Nuvem

Pipeline de Automação no GitHub Actions (.github/workflows/ci.yml):

- **Job 1 — Qualidade dos Dados:** Executa `npm run validate-data`, auditando formatos, chaves nulas e conflitos;
- **Job 2 — Testes Automatizados:** Executa `npm test` (13 testes automatizados cobrindo fórmulas, limites e regras de negócio);
- **Job 3 — Verificação de Segurança (DevSecOps):** Realiza busca automatizada para impedir o versionamento de arquivos `.env` com credenciais reais;
- **Job 4 — Compilação e Build:** Valida a compilação do front-end React para garantir ausência de erros em produção.

Arquitetura em Nuvem Híbrida:

Camada	Tecnologia / Provedor	Função Arquitetural	Evidência / Configuração
Front-End SPA	Vercel (PaaS / Edge Network)	Hospedagem estática com CDN global e SSL automático	`APP/font_end/vercel.json`
Banco de Dados	Supabase (PostgreSQL 15 Nuvem)	Persistência relacional gerenciada com RLS e backups	`APP/Banco_de_dados/schema-relacional.sql`
API RESTful	Node.js & Express 5	Regras de negócio, cálculo de indicadores e healthcheck	`APP/beck_end/server.js`
Ambiente Local	Docker & Docker Compose	Execução local 100% determinística e reproduzível	`docker-compose.yml`

Observabilidade e Healthcheck:

O endpoint oficial `GET /api/health` retorna:

```
{ "ok": true, "storage": "supabase", "timestamp": "2026-11-03T22:30:00Z" }
```

Roteiro de Rollback e Recuperação de Falhas (Runbook):

- **Rollback Front-End (Vercel):** Acesso à aba *Deployments* → *Instant Rollback* (restaura a versão anterior estável em menos de 10 segundos);
- **Rollback de Código (Git):** Reversão atômica via `git revert <commit-hash>` e novo push para disparar a esteira;
- **Recuperação de Banco de Dados:** Painel do Supabase com restauração de snapshot (Point-in-Time Recovery).

5. Extensão Opcional — Automação Industrial e Telemetria IoT (Bônus)

Para o cumprimento da extensão opcional com bônus de pontuação, o projeto implementou o subsistema de sensoriamento predial inteligente:

- **Sensores Simulados:** Presença Infravermelha (PIR HC-SR501), Temperatura/Climatização (DHT22) e Corrente Elétrica (SCT-013);
- **Simulador Autônomo:** Script `automation/simulador_sensores.js` produz leituras em tempo real e emite alertas automáticos caso a temperatura exceda 26.5°C com a sala desocupada;
- **Segurança Operacional:** Operação estritamente isolada de máquinas de alta tensão, sem risco físico ou quebra do fluxo acadêmico principal.

6. Roteiro de Demonstração Rápida para os Avaliadores

```
# 1. Validação de dados e testes automatizados na raiz do repositório: npm run ci # 2. Execução da simulação de telemetria IoT de sensores: node automation/simulador_sensores.js # 3. Subida do ecossistema local reproduzível via Docker: docker compose up -d # 4. Verificação de saúde da API: curl http://localhost:3001/api/health
```